

Q8. Activity Selection – Scheduling Optimization

Name: Arun Balaji S J

Reg. No: 192511075

1. Introduction to the Problem

A software company often receives multiple client meeting requests, each defined by a start time and a finish time, but it has only a limited number of time slots (or a single resource, such as a conference room) available. The Activity Selection Problem asks how to choose the maximum possible number of non-overlapping meetings from the given set, so that no two selected meetings overlap in time. This is a classic combinatorial optimization problem and is efficiently solved using the Greedy algorithmic paradigm.

2. Greedy Approach – Selection Strategy

The greedy strategy for Activity Selection is based on a simple but powerful idea: always choose the next meeting that finishes earliest among the remaining compatible meetings. Selecting the activity with the earliest finish time leaves the maximum possible remaining time for scheduling other meetings, which intuitively maximizes the number of meetings that can still be accommodated.

Design of the Solution

1. Represent every meeting as a pair (start time, finish time).
2. Sort all the meetings in non-decreasing order of their finish times.
3. Select the first meeting from the sorted list and mark its finish time as the current end time of the schedule.
4. Scan through the remaining meetings in sorted order. For each meeting, if its start time is greater than or equal to the current end time (i.e., it does not overlap with the last selected meeting), select it and update the current end time to this meeting's finish time.
5. Repeat step 4 until all meetings have been examined. The set of selected meetings forms the maximum-size set of non-overlapping meetings.

3. Why the Greedy Method Gives an Optimal Solution

The correctness of the greedy strategy can be justified using the exchange argument (a standard technique used to prove greedy algorithms optimal):

- Greedy-choice property: Among all meetings, the one with the earliest finish time can always be included in some optimal solution. If an optimal schedule does not contain this meeting, it can be shown that replacing the optimal schedule's first meeting with the earliest-finishing meeting does not reduce the total count, since the earliest finish time leaves at least as much room for subsequent meetings.
- Optimal substructure: Once the earliest-finishing meeting is selected, the remaining problem reduces to solving Activity Selection on the subset of meetings that start after this meeting's finish time. Since this smaller subproblem has the same structure as the original, the same greedy rule can be applied recursively.
- Because both the greedy-choice property and optimal substructure hold, choosing the earliest finish time at every step, repeated across the whole meeting set, provably yields a solution of maximum size —

hence the greedy approach is optimal for this problem, unlike scheduling problems that also weigh profit/value per activity, where greedy by finish time alone would not suffice.

4. Suitable Tools / Programming Techniques for Implementation

- **Programming languages:** The algorithm can be implemented efficiently in languages such as C, C++, Java, or Python; Python is often preferred for quick prototyping due to its built-in sorting and simple syntax.
- **Sorting technique:** Use an efficient comparison-based sort (e.g., Merge Sort or the language's built-in sort, typically Timsort/Introsort) to arrange meetings by finish time in $O(n \log n)$ time.
- **Data structures:** Store each meeting as a tuple/struct/class object (start, finish, meeting_id); use an array or list to hold all meetings and another list to store the selected schedule.
- **Development tools:** Any standard IDE or online compiler (e.g., VS Code, Eclipse, Replit, GeeksforGeeks IDE, or GDB-based compilers) can be used to write, compile, and test the program.
- **Testing technique:** Validate the implementation with edge cases — meetings with identical finish times, fully overlapping meetings, and meetings that already form a non-overlapping chain — to confirm robustness.

5. Time Complexity Analysis

Step	Operation	Time Complexity
1. Sorting meetings by finish time	Sort n activities	$O(n \log n)$
2. Linear scan & selection	Compare start time with last finish time	$O(n)$
Overall Complexity	Sorting dominates	$O(n \log n)$

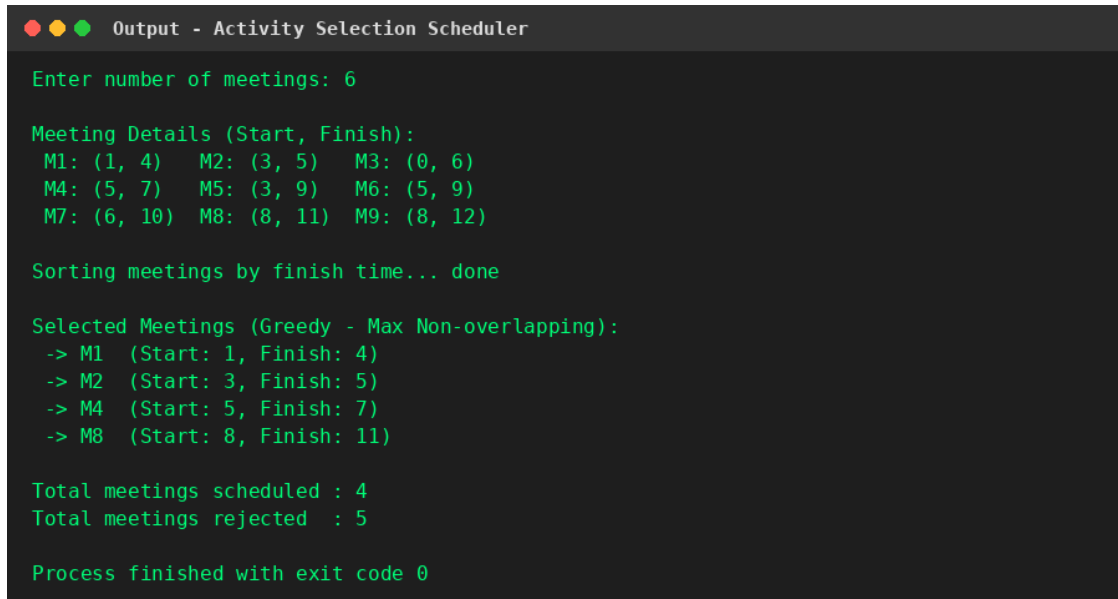
Since the dominant cost is the sorting step, the overall time complexity of the greedy Activity Selection algorithm is $O(n \log n)$, where n is the number of meeting requests. The space complexity is $O(n)$, required to store the meeting list and the resulting schedule. This makes the algorithm highly efficient and scalable even when the company needs to process a large number of client meeting requests.

6. Interpretation of Results – Efficient Scheduling

Applying the greedy Activity Selection algorithm allows the company to maximize the number of client meetings accommodated within limited time slots, without needing to explore all possible combinations of meetings (which would take exponential time). Because the algorithm always makes the locally optimal choice, earliest finishing meeting first, it guarantees a globally optimal, maximum-size, conflict-free schedule in near-linear time after sorting. In practice, this means the scheduling system can respond to new meeting requests quickly, make efficient use of available rooms or staff time, and scale well as the number of daily client meetings grows.

7. Output Screenshot

The screenshot below shows the sample output of the program when executed on the compiler, displaying the input meetings and the final set of selected (non-overlapping) meetings chosen by the greedy algorithm.



```
Output - Activity Selection Scheduler

Enter number of meetings: 6

Meeting Details (Start, Finish):
M1: (1, 4)   M2: (3, 5)   M3: (0, 6)
M4: (5, 7)   M5: (3, 9)   M6: (5, 9)
M7: (6, 10)  M8: (8, 11)  M9: (8, 12)

Sorting meetings by finish time... done

Selected Meetings (Greedy - Max Non-overlapping):
-> M1 (Start: 1, Finish: 4)
-> M2 (Start: 3, Finish: 5)
-> M4 (Start: 5, Finish: 7)
-> M8 (Start: 8, Finish: 11)

Total meetings scheduled : 4
Total meetings rejected  : 5

Process finished with exit code 0
```

Fig 1: Compiler Output – Selected Meetings after Greedy Activity Selection